



THE NCUK INTERNATIONAL FOUNDATION YEAR

IFYCO002 Computer Science

Coursework

2025-26

Coursework assessment includes content from Learning Outcomes LO4 and LO7 of the syllabus and may have overlapping content from other topics.

Learners will submit their coursework online via NCUK's assessment platform. All assessments will be subject to a plagiarism check using Turnitin.

Teachers at the Study Centre will be responsible for marking coursework in accordance with the NCUK processes and policies.

Context Choices

The coursework contains two projects – a Python project and a database project. You must complete both projects submitting a written report for both.

A scenario has been provided to set each problem in a real-world context.

The projects are a substantial piece of work which assesses a variety of different skills including the development and demonstration of computational thought processes and database design. The following guidance should be considered before learners embark on this assessment.

- ⇒ Students have access to appropriate database software and a programming language (which is non-trivial and has a substantial coded element).
- ⇒ Centre software and resources allow students to fully solve the set problems sensibly within the constraints of resources available.
- ⇒ Involve all elements of the skills of analysis, design, development and evaluation.
- ⇒ Students are equipped with the skills in screen-shotting evidence, cropping images for suitability and annotating evidence.

Project documentation structure

Candidates are encouraged to use the NCUK coursework template, which provides structured guidance on what to include at each stage of the report. The template is provided in PowerPoint format to support ease of use, particularly for adding screenshots, annotations, and labels. While this format is not industry standard for a report, it has been selected to simplify the creation process for learners and marking process for centres.

Centres may adopt an alternative structure if preferred, but it must still allow candidates to fully demonstrate all assessment requirements.

Independence

While teachers may provide general guidance and support, each candidate's submission should reflect their own individual work. The solution should be independently designed and implemented by the student. To ensure fairness and authenticity, whole-class teaching must avoid leading learners toward identical outcomes.

Assessment Criteria

Learners are expected to demonstrate their ability to design, develop, test and evaluate their solutions in each project. The problem must be solvable effectively in the chosen language, so it is important there is a good match between the requirements of the problem and the language that is written in Python.

Data Files

A prebuilt 2D array is provided for the Python task (rooms.py)

To maximise learning time and minimise repetitive data entry, NCUK has provided pre-prepared CSV files for students to import into their database structures. Some of these files intentionally do not include header rows, allowing students to define and apply their own field names as part of the implementation process.

Students are not required to delete, amend, or add to the data unless instructed to do so within a specific task.

NCUK also provide .png image files for incorporating into business reports and forms. All data and images within the datasets are AI-generated. Any resemblance to real individuals, companies, or organisations is purely coincidental.

Design Assessment Criteria (10 marks total)

Design marks are awarded for how clearly and effectively the student has planned their solution. This includes the structure, logic, naming conventions, choice of tools or methods, and how well the proposed solution addresses the problem before development begins. Design evidence may include flowcharts, pseudocode, ER diagrams, table designs, field justifications, or written rationale.

0-3 marks	4-6 marks	7-10 marks
A limited or incomplete design is presented, with minimal evidence of planning. Key components are missing, unclear, or incorrect. Diagrams or pseudocode may be poorly constructed or absent. The approach lacks structure and does not clearly show how the task will be tackled. The design does not reflect the client requirements or is generic in nature.	The design shows a reasonable attempt to plan the solution, but with some missing detail or unclear logic. Some key features or structures are identified, but not always fully developed or justified. Naming conventions and structure are present but may be inconsistent or basic. Diagrams are included but may be limited in clarity or accuracy. There is partial connection to the client brief, but the design may not fully reflect all requirements.	The design is clear, detailed, and shows a well-thought-out approach to solving the task. Key components are planned thoroughly, with correct use of conventions (e.g. clear variable/field names, logical process flow, well-structured data relationships). There is strong alignment between the design and the client requirements, showing understanding of the problem. Diagrams such as flowcharts or ERDs are accurate, well-labelled, and clearly support the proposed solution. There is clear evidence of independent thinking and appropriate tool selection.

Implementation Assessment Criteria (20 marks total)

Implementation marks are awarded for the quality, completeness, and accuracy of the final product. This includes how well the program or database meets the task requirements, the use of appropriate syntax or structure, and the overall standard of the finished solution. Implementation must be the student's own work, independently created, and clearly documented.

0-5 marks	6-10 marks	11-15 marks	16-20 marks
An incomplete or mostly non-functional solution is submitted. The implementation lacks structure, and only a small part of the task has been attempted. There is little evidence of understanding of the programming language or database system. Major errors are present, and outputs (if any) are not meaningful.	The solution is partially working or only meets a few aspects of the task. There may be significant errors, missing features, or structural issues that impact functionality. The implementation may lack clarity, with poor or inconsistent syntax or logic. The output is often incorrect or only partially useful. Understanding of the tool is basic, and the solution may require extensive supervisor support to get running.	The solution works well and covers all or nearly all task requirements. There may be minor errors or small inefficiencies, but the program or database performs as intended overall. Code or queries are mostly well written and logical. There is evidence of sound understanding of the tools used, even if not all advanced features are fully implemented.	The solution is fully functional, complete, and meets all client requirements. It runs without errors and produces accurate, relevant results showing strong evidenced of good programming. There is clear evidence of advanced techniques being implemented. Code or query structure is clean, well-organised, and easy to follow. The final product reflects a high level of independent technical skill and confidence in the chosen tools.

Testing Assessment Criteria (15 marks total)

Testing marks are awarded based on the range, quality, and accuracy of the testing carried out. Students must provide evidence that their solution has been tested appropriately, using a variety of inputs and conditions. This may include test tables, screenshots, and analysis of results. The focus is on both the execution of tests and the understanding of what the results show.

0-5 marks	6-10 marks	11-15 marks
Minimal testing has been attempted. Tests may be basic, poorly recorded, or limited in range. Evidence is unclear or missing for several parts of the system. There is little or no analysis of test results, and limited understanding of the purpose of testing.	Some testing has been carried out, covering the main functionality of the solution. There is evidence of different types of test cases, though not all may be relevant or clearly explained. Documentation may lack detail or structure in places, and analysis of results may be limited. There is some understanding of testing, but the approach could be more thorough.	A wide range of tests has been carried out, including normal, boundary, and invalid cases. Testing is well structured and clearly documented using logs, tables, or screenshots. Results are analysed accurately, with clear explanations of what worked, what didn't, and why. The student reflects on the effectiveness of the solution and may identify improvements based on test outcomes.

Evaluation Assessment Criteria (5 marks total)

Evaluation marks are awarded based on how well the student reflects on the effectiveness of their solution. This includes how well it meets the client's requirements, its performance, limitations, and suggestions for future improvement.

0-3 marks	4-5 marks
Some evaluation is attempted, but it may be brief, vague, or overly general.	The student gives a clear and thoughtful evaluation of their solution.
There is limited reference to the original requirements or little discussion of the solution's effectiveness.	They comment on how well it meets the brief, identify strengths and weaknesses, and explain any limitations.
Improvements may be mentioned but are not clearly explained or linked to specific issues.	Suggestions for improvement or optimisation are relevant and well justified.
	The evaluation shows a good level of understanding and self-reflection.

Task A Python Task

Northbridge University Accommodation Services Coursework Brief 2025-26

You should submit a written report of 1000 words approximately which covers the following tasks.

Background

The Accommodation Office at Northbridge University needs a simple Python program to manage and allocate student rooms based on key criteria. You will build a menu-driven application using a list of rooms stored in a 2D array. Each room is either available ('EMPTY') or already occupied.

You have been tasked with designing, building, and testing a modular Python application that can handle this room allocation process.

Project Overview

You will build a program that:

- Collects student details
- Uses their stored details to search for suitable available rooms
- Stores new room allocations
- Displays a summary of allocations and calculates vacancies

Data file

You will be provided with starter code `rooms.py` which contains the room database stored as a 2D array. Each entry contains the name of the Hall, the room ID, who currently occupies it and whether it is a ground floor or not.

'EMPTY' indicates the room is available to be allocated.

Example:

```
rooms = [  
    ["Maple Hall", "MH101", "EMPTY", True],  
    ["Oak Hall", "OH202", "Ali Khan", False]  
]
```

#This is room 101 in Maple Hall, currently empty on the ground floor, room 202 in Oak Hall but is currently occupied by Ali Khan and is not suitable for accessibility requirements

Task A1: Design (10 marks)

Give a brief overview of the problem and justify why Python is a suitable tool for the solution.

Create a design of your solution with explanations that justify your design choices.

Task A2: Build the Program (20 marks)

Your program must include the following menu options:

Option Description

1	Enter new student details (name, subject studying, year of study (1, 2 or 3) and accessibility requirements)
2	List suitable rooms based on the rules
3	Allocate a student to a chosen room
4	Display all room allocations and vacancies
0	Exit the program

In Option 2, the user receives a list of the available rooms based on the matching rules. The user will manually select a room ID from the list they believe to be the best choice.

In Option 3, they use the stored student name and the relevant RoomID to make an allocation. The program does not automatically allocate rooms.

Room Matching Rules

Rule	Condition
Student has accessibility requirements	<i>Must be ground floor</i>
Student is in Year 1	<i>Avoid Maple Hall</i>
Studies Law	<i>Prefer Oak Hall</i>

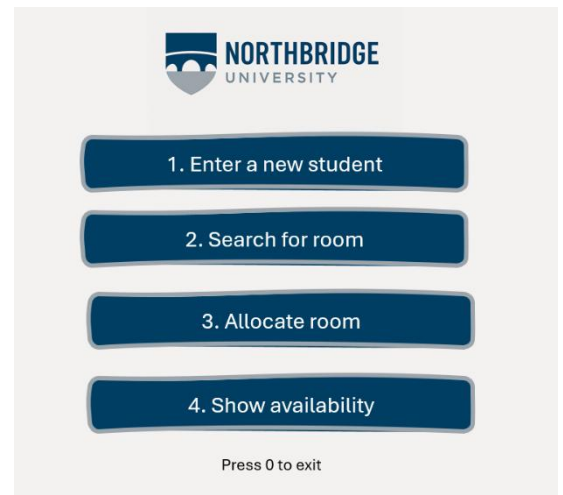
Suitable helpful messages should be displayed throughout the program. Your code should use good programming techniques throughout including maintainability.

The evidence you provide should be annotated to explain the key decisions you have taken during implementation.

Independent Technical Skill

To achieve the highest marks for implementation you must also choose to increase the functionality of the program. Here are a range of suggestions but you may come up with your own independent options:

- i. Allow staff to add a new room or delete a room from the rooms data store
- ii. Allow staff at end of the year to reset all rooms back to empty
- iii. Save new allocations to a .txt or .csv file
- iv. Create a menu driven interface like the one shown opposite
- v. Students in Oak Hall pay £1400 a year, other students pay £1150 – calculate the current income from the occupied rooms.



Task A3: Testing (15 marks)

Create and implement a testing strategy for your program to include:

- ⇒ Tests for all aspects of the program.
- ⇒ A range of test data that includes the test data below
- ⇒ How the program has been put through unit testing
- ⇒ How the program will be tested as a whole

Show evidence of testing your solution, include screenshots and explanations.

Test Data:

name	Matthew Adams
subject	Computing
year of study	1
room type	En-suite
accessibility needs	True

Task A4: Evaluation (5 marks)

Write an evaluation covering:

- ⇒ Client requirements – to what extent does your program do what was asked in the project overview?
- ⇒ Code efficiency and structure – is it easy to maintain?
- ⇒ Limitations – what didn't work as expected?
- ⇒ Scalability – could your system scale up to cope with more users, rooms and students?

Task B: Database Task

Northbridge University Accommodation Services

Coursework Brief 2025-26

You should submit a written report of 1000 words approximately which covers the following tasks.

Background

The University Accommodation Services (UAS) team at Northbridge University in London manages all on-campus student housing. Each year, thousands of students apply for accommodation, and the department requires a reliable, efficient database system to manage room allocations, tenancy records, and maintenance requests.

Project Overview

You have been hired as a junior database developer to help UAS replace their outdated spreadsheet system with a relational database. This system must store data about students, halls of residence, rooms, tenancies, and maintenance issues.

Requirements

Data to be stored must include:

- Students: Full name, student ID, e-mail and phone number, subject of study, year of study, and accessibility needs.
- Halls of Residence: Hall name, location, and total capacity.
- Rooms: Room number, type (e.g. single or en-suite), accessibility, and price per week.
- Tenancies: Which student is assigned to which room and for which academic year.
- Maintenance Requests: Student ID, room ID, issue description, date raised, status (Pending or Resolved), and assigned staff member (BMC or SJL).

Key Relationships

- Each hall contains multiple rooms.
- A student can only live in one room per academic year.
- A room can only be assigned to one student per tenancy period.
- A student may submit multiple maintenance requests for their room.

Data Files Provided

Your tutor will provide the following files:

- students.csv, rooms.csv, halls.csv, tenancies.csv & maintenance Requests.csv

Note: Students and Maintenance files do not include field names

Task B1: Design (10 marks)

Briefly describe the scenario and explain why a relational database is more appropriate than a spreadsheet.

Create a fully normalised Entity Relationship Diagram (ERD), showing:

- Entities, attributes, data types, primary/foreign keys, and relationships.

Provide justification for your design choices and relationships.

Task B2: Implementation (20 marks in total – Tasks I - VI)

Task B2 i) Setting up the Database

Set up the Database

- Import CSV files and define table structures.
- Add suitable field properties and formats.
- Set up sensible validation rules on the Students table only

Collect and label screenshots of the design and datasheet views.

Task B2 ii) Implementation - SQL

Use SQL to:

- UPDATE the price of room H2R104 to 140.
 - DELETE all maintenance requests that contain "WiFi issue".
 - INSERT a new student record:
(2045, 'Nida Lim', 'nida.lim@northbridge.ac.uk', '07911123456', 1, 'Wheelchair Access', 'Business Management')
-

Task B2 iii) Implementation - Queries

Implement a solution to any THREE of the following queries, those that are marked (*) are more difficult and any marked (**) require independent technical skill.

Choose any 3 from:

- A list of students with accessibility needs
 - A list of vacant rooms in Maple Hall
 - A list of ground floor rooms
 - A list of Computing students and the hall they are staying in (*)
 - A list of pending maintenance issues with room and student (*)
 - A list of old maintenance issues (14+ days) (*)
 - A report of the average room price per hall with a list of rooms priced above the average (**)
-

Task B2 iv) Create a Form

Create a form for entering new students using the Northbridge University house style:

- Colours: Light grey (#F2F2F2), Dark blue (#003366)
- Font: Calibri

Task B2 v) Creating a Report

Generate a report for staff member Sharon Lloyd (SJL) showing all maintenance jobs assigned to her this week. Group the report to maximise her efficiency.

Task B2 vi) Responding to Email Request

Use your solution to draft a reply to your manager Dana:

Hi,

We've just had Justin Han-Soe in the office.

He's a returning third-year Business Management student. He's requested a room on North Campus, preferably with other Year 3 students on the same course but not on the ground floor. Do we have any available rooms that meet his requirements?

Dana

Task B3: Testing (15 marks)

Carry out a thorough testing process for your solution. You must:

- Perform a Range of Tests
- Analyse the Results

Task B4: Evaluation (5 marks)

Write an evaluation covering:

- Does the database meet the universities' needs?
- Are queries effective and efficient?
- What are the remaining problems with your solution and how could these be fixed?
- Suggested future improvements?

Submission Requirements

- Python:
 - A PDF document including evidence of tasks A1 – A4
 - Your final .py file
- Databases:
 - A PDF document including evidence of tasks B1 – B4
- Clear labels, commentary, and evaluation.
- File formats accepted: PDF, Word, Powerpoint

Make sure your work is clearly presented, well tested, and thoroughly explained. Your work will be checked against plagiarism, including use of AI, therefore make sure the solutions you submit are your own.
